Two Pointers:

When to use)

- linear data structures (array, string, list)
- when you need to scan the start & end of a list
- finding pairs in a sorted list
- removing duplicates / filtering

What is it)

- instead of scanning for all possible substrings / subarrays, use two pointers i and j at each end of the string or sorted array.

* different from sliding window b/c pointers are at each end, rather than moving together up or down an array. SQUEEZE not SLIDE

Why use it)

- will lower time complexity from $O(n^2) \rightarrow O(n)$

racecar

i j

- checking for palindrome by moving pointers in from both sides of a valid (% 2 == 0) string, checking if the chars are equal.

```
i = 0
j = len(input) - 1

while i < j :
    if condition:
        adjust i
    else
        adjust j
```

Slow & Fast Pointers:

When to use)

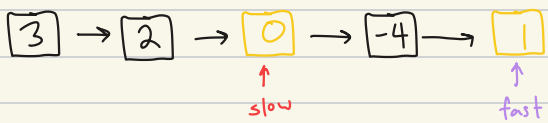
- linear data structures
- detect cycle in linked list
- find middle of linked list
- perform in one pass w/ O(1) space

What is it)

- use two pointers, a slow and fast pointer. Slow moves moves once and fast moves twice at every iteration.

Why use it)

- Instead of using a data structure to hold previous nodes to detect a cycle O(n), using this technique will find it with O(1) space if fast loops around and eventually finds slow.



} found the end while slow found the middle

```
slow = head
fast = head

while fast and fast.next:
    slow = slow.next #move once
    fast = fast.next.next #move twice

    update based on custom logic.
```